Vježba 4 - detekcija linija, detekcija kružnica, maskiranje slike, video petlja, detekcija linija voznih traka u videu

Sadržaj

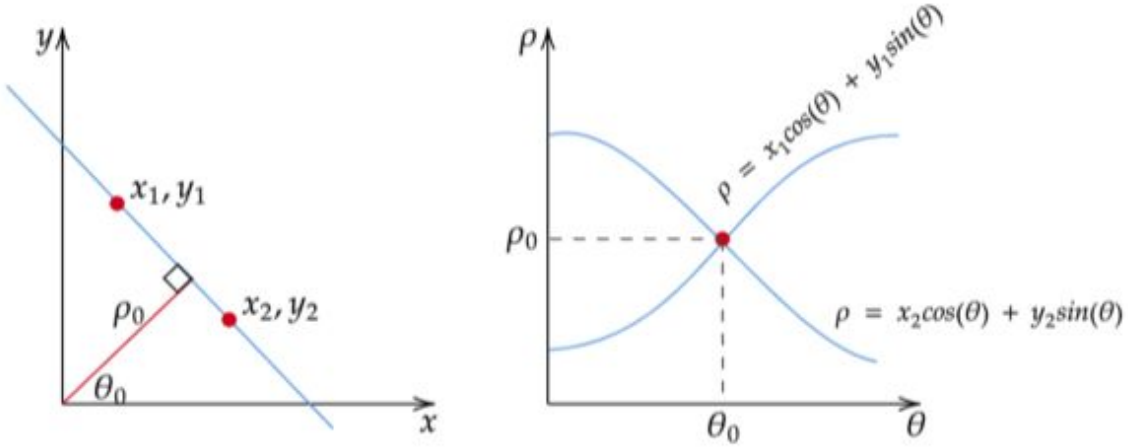
- Detekcija linija
  - cv2.HoughLines() i cv2.HoughLinesP()
- Detekcija kružnica
  - cv2.cv2.HoughCircles()
- Maskiranje slike
- Video petlja
- Detekcija linija voznih traka u videu

Detekcija linija

Detekcija linija

- Houghova transformacija - tehnika za detekciju bilo kojeg oblika čak ako on i nije potpun ili ako postoji određeno prekidanje
- Ulaz - binarna slika koja sadrži detektirane rubove
  - Obično se koristi Canny detektor rubova
- Parametarski prostor (ρ, θ)
  - ρ - udaljenost pravca od ishodišta
  - θ - kut nagiba normale pravca s obzirom na *x* os
- Traži se skupina "glasova" u akumulatoru
- OpenCV nudi dvije funkcije za detekciju linija Houghovom transformacijom

1. cv2.HoughLines()
2. cv2.HoughLinesP()



Detekcija linija

<u>“Standardna” Houghova transformacija</u>

cv.HoughLines(image, rho, theta, threshold) -> lines

- image - 8-bitna jednokanalna binarna slika
  - slika koja sadrži rubove - obično nakon Cannya
- rho - razlučivost ρ u pikselima (npr. 1)
- theta - razlučivost θ u radijanima (npr. np.pi/180)
- threshold - minimalan broj glasova u akumulatoru da bi se linija prihvatila

Izlaz

- lines - lista oblika [[rho, theta], …] shape -> (broj_linija, 1, 2)
- Svaka linija zadana je u polarnoj formi $x\cos\theta+y\sin\theta=\rho$

Detekcija linija

<u>Probabilistička Houghova transformacija</u>

cv.HoughLinesP(image, rho, theta, threshold[, **minLineLength**[, **maxLineGap**]]) -> lines

- image - 8-bitna jednokanalna binarna slika
  - slika koja sadrži rubove - obično nakon Cannya
- rho - razlučivost ρ u pikselima (npr. 1)
- theta - razlučivost θ u radijanima (npr. np.pi/180)
- threshold - minimalan broj glasova u akumulatoru da bi se linija prihvatila
- **minLineLength** - minimalna dužina segmenta da bi se linija prihvatila
- **maxLineGap** - maksimalni razmak između točaka koji se "može spojiti" u jedan segment

Izlaz

- lines - lista oblika [[x1, y1, x2, y2], …] shape -> (broj_linija, 1, 4)

Detekcija linija

Algoritam detekcije linija Houghovom transformacijom

1. Učitavanje slike u nijansama sive boje
2. Primjena Canny detektora rubova
3. Primjena Houghove transformacije za detekciju linija
4. Crtanje detektiranih linija

Detekcija linija

- Prikaz detektiranih linija pomoću funkcije cv2.line()

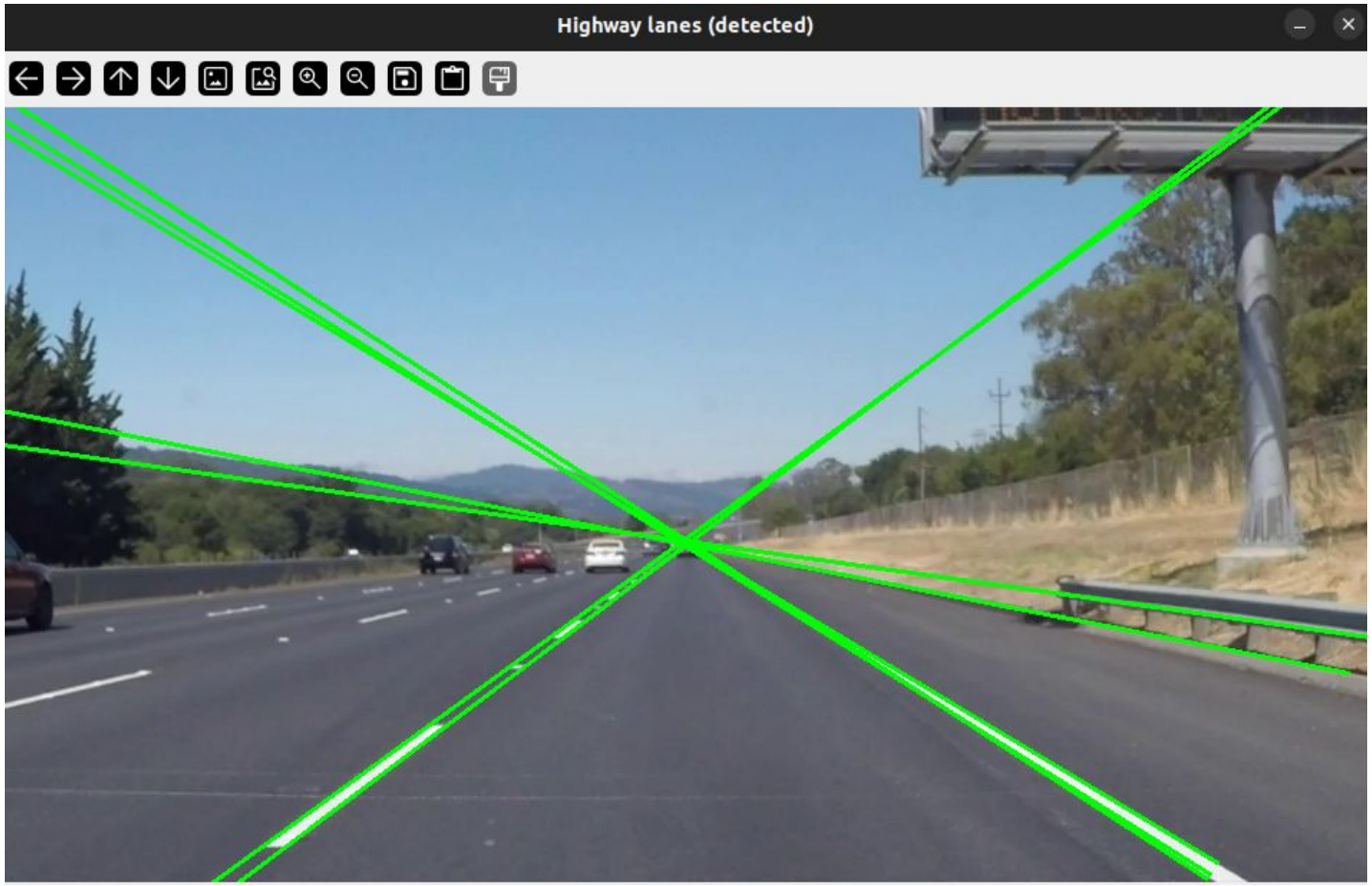
cv2.line(img, pt1, pt2, color, thickness)

- img - slika na kojoj će se prikazati linija
- pt1 - početna točka
- pt2 - krajnja točka
- color - boja linije
- thickness - debljina linije

Detekcija linija

Primjer - Detekcija linije - Standardna Houghova transformacija

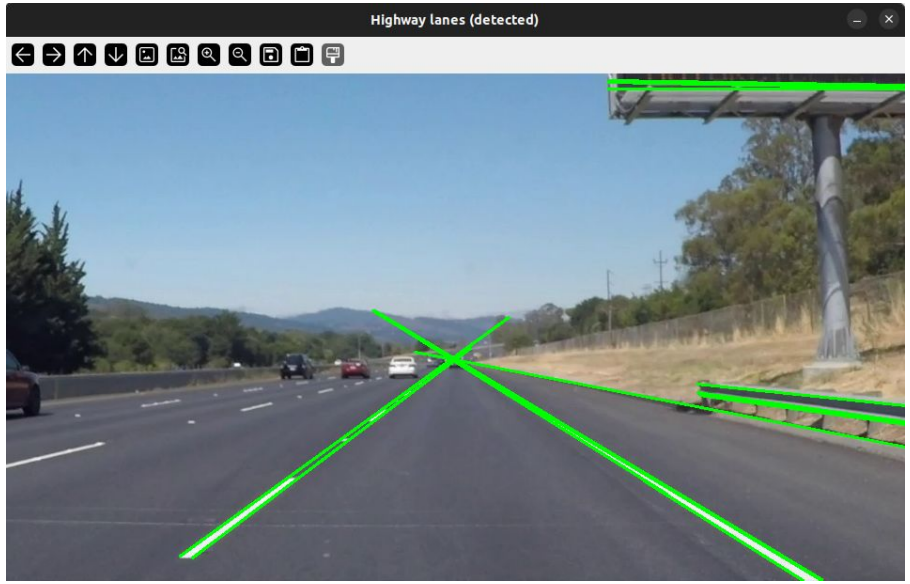
```python
import cv2
import numpy as np

image_bgr = cv2.imread("lanes.jpeg", cv2.IMREAD_COLOR)
# 1. Ucitavanje slike u nijansama sive boje
image_gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)

# 2. Primjena Canny detektora rubova
image_canny = cv2.Canny(image_gray, 100, 200)

# 3 Primjena Houghove transformacije za detekciju linija
lines = cv2.HoughLines(image_canny, rho=1, theta=np.pi / 180, threshold=180)

# 4. Crtanje detektiranih linija
if lines is not None:
    for line in lines:
        rho, theta = line[0]
        a = np.cos(theta)
        b = np.sin(theta)
        x0 = a * rho
        y0 = b * rho

        x1 = int(x0 + 1000 * (-b))
        y1 = int(y0 + 1000 * (a))
        x2 = int(x0 - 1000 * (-b))
        y2 = int(y0 - 1000 * (a))
        cv2.line(image_bgr, (x1, y1), (x2, y2), (0, 255, 0), 2)
else:
    print("No lines were found by HoughLines.")

cv2.imshow("Highway lanes (detected)", image_bgr)
cv2.waitKey()
cv2.destroyAllWindows()
```



Detekcija linija

Primjer - Detekcija linije - Probabilistička Houghova transformacija

```python
import cv2
import numpy as np

image_bgr = cv2.imread("lanes.jpeg", cv2.IMREAD_COLOR)
# 1. Ucitavanje slike u nijansama sive boje
image_gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)

# 2. Primjena Canny detektora rubova
image_canny = cv2.Canny(image_gray, 100, 200)

# 3 Primjena Houghove transformacije za detekciju linija
lines = cv2.HoughLinesP(image_canny, rho=1, theta=np.pi / 180, threshold=150, minLineLength=100, maxLineGap=100)

# 4. Crtanje detektiranih linija
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(image_bgr, (x1, y1), (x2, y2), (0, 255, 0), 2)
else:
    print("No lines were found by HoughLinesP.")

cv2.imshow("Highway lanes (detected)", image_bgr)
cv2.waitKey()
cv2.destroyAllWindows()
```



Detekcija linija

Usporedba

Značajka	**Standardna Hough**	**Probabilistička Hough**
Uzorkovanje	Sve rubne točke	Samo dio rubnih točaka (nasumično)
Rezultat	ρ–θ (beskonačne linije)	Segmenti linija (x1, y1, x2, y2)
Brzina	Sporija	Brža
Praktičnost	Za analizu	Za primjene (posebno u realnom vremenu)
Fino podešavanje	Malo parametara	Više parametara za prilagodbu

Standardna Houghova traži linije teorijski – gleda sve moguće točke i vraća savršenu, beskonačnu liniju.
Probabilistička Houghova traži linije praktično – gleda samo dio točaka i vraća konkretne segmente koje vidi u slici.

Detekcija linija

Kako bismo odredili nagib detektirane linije? Je li linija okomita, vodoravna, rastuća ili padajuća?

numpy.polyfit(x, y, deg)

- x -> proslijediti x1 i x2
- y -> proslijediti y1 i y2
- deg -> 1 (prvi stupanj odnosno linija)

Povratna vrijednost:

- Parametri linije:
  - **nagib pravca**
  - odsječak na osi y

Detekcija kružnice

Detekcija kružnice

$$(x - x_{center})^2 + (y - y_{center})^2 = r^2$$

cv2.HoughCircles(image, method, dp, minDist, param1, param2, minRadius, maxRadius)

- image - ulazna slika u nijansama sive boje
- method - metoda detekcije (cv2.HOUGH_GRADIENT je jedina dostupna)
- dp - inverz rezolucije akumulatora. za dp=1 rezolucija akumulatora jednaka je rezoluciji slike. Za dp=2 rezolucija je duplo manja
- minDist - minimalna udaljenost središta susjednih kružnica
- param1 - veća vrijednost praga tijekom Canny detekcije rubova
- param2 - prag akumulatora
- minRadius - minimalni polumjer
- maxRadius - maksimalni polumjer

Detekcija kružnice

- cv2.HoughCircles vraća detektirane kružnice u formatu centar_x, centar_y, polumjer
- Kako bi prikazali detektirane kružnice na originalnoj slici, možemo koristiti funkciju cv2.circle()

cv2.circle(img, center, radius, color)

- img - slika na kojoj će se prikazati kružnica
- center - koordinate središta kružnice
- radius - polumjer kružnice
- color - boja kružnice
- thickness - debljina kružnice

Detekcija kružnice

Napomena:

- cv2.HoughCircles() -> povratne vrijednosti su tipa float
- Ako su nam potrebni cjelobrojni tipovi, možemo koristiti:
  - np.uint16(np.around(circles))

Detekcija kružnice

Primjer - Detekcija kružnice

```
import cv2
import numpy as np

slika = cv2.imread('kovanice.png')
grayscale_slika = cv2.cvtColor(slika, cv2.COLOR_BGR2GRAY)
grayscale_slika_blurred = cv2.GaussianBlur(grayscale_slika, (7, 7), 0)
circles = cv2.HoughCircles(grayscale_slika_blurred, cv2.HOUGH_GRADIENT, 1, 1, param1=300, param2=90)
circles = np.uint16(np.around(circles))

for circle in circles.squeeze():
    cv2.circle(slika, (circle[0], circle[1]), circle[2], (0, 255, 0), thickness=3)

cv2.imshow("circles", slika)
cv2.imshow("circles blurred", grayscale_slika_blurred)
cv2.waitKey()
cv2.destroyAllWindows()
```

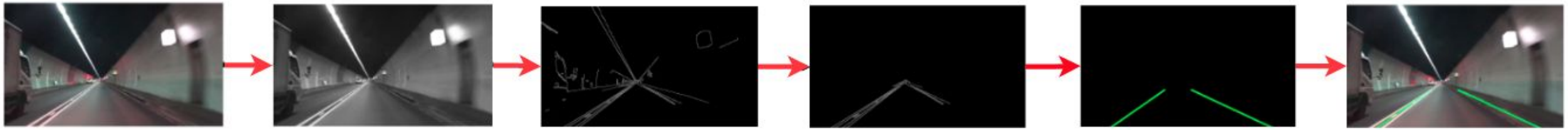
Maskiranje slike

Maskiranje slike

- Prilikom detekcije objekata ponekad nas zanima samo specifičan dio slike
  - desna polovica -> tamo se obično nalaze prometni znakovi iz perspektive vozača
  - u “trokutastom” području donje polovice slike s kamere obično se nalaze linije koje obilježavaju vozne trake

Maskiranje slike

- Maskiranje slike je često korištena tehnika u procesu detekcije objekata u određenom području slike



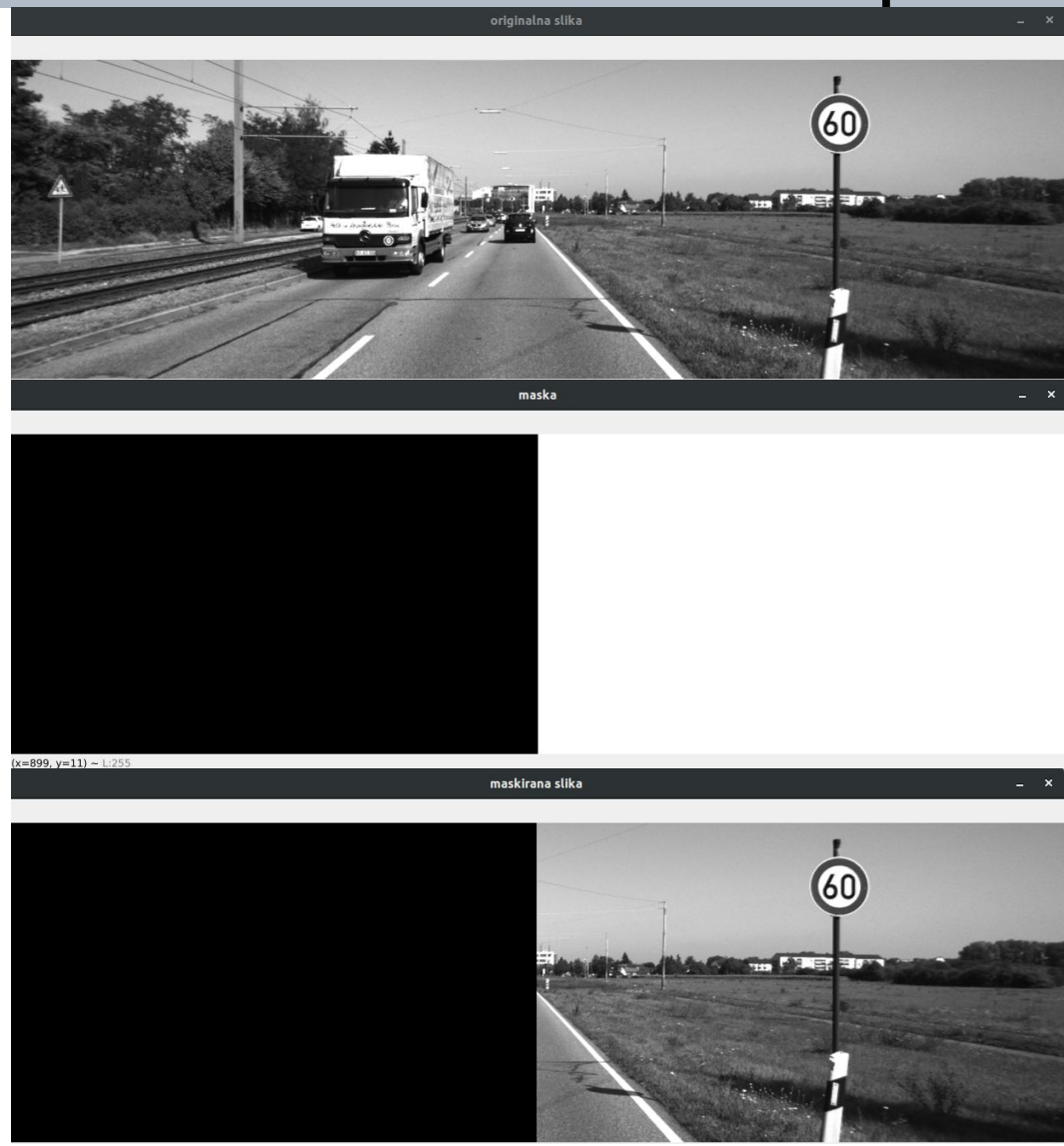
Originalna slika → Slika u sivim nijansama → Canny → Primjena maske → Detekcija linija → Prikaz detekcije na originalnoj slici

Izvor: https://medium.com/analytics-vidhya/building-a-lane-detection-system-f7a727c6694

Maskiranje slike

Primjer - Maskiranje slike - fillPoly

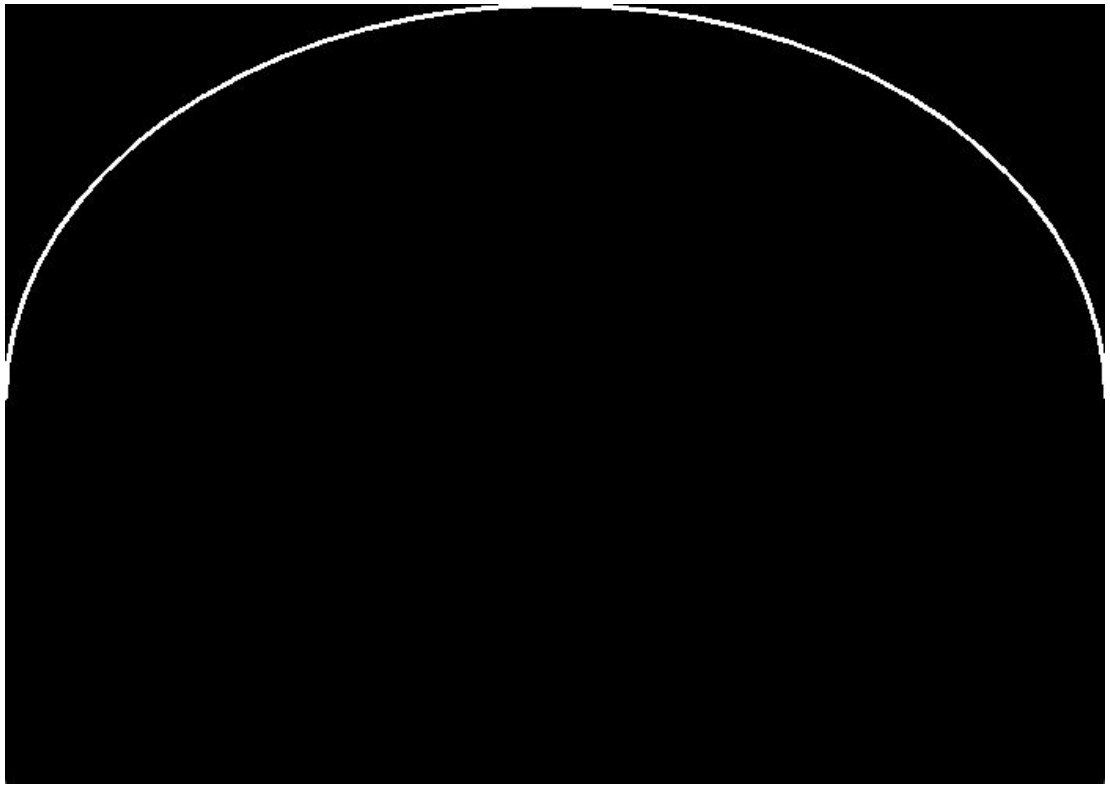
```
import cv2
import numpy as np

im = cv2.imread('kamera4.png', cv2.IMREAD_GRAYSCALE)
mask = np.zeros_like(im)
height, width = im.shape

x1 = width / 2
y1 = 0
x2 = width
y2 = 0
x3 = width
y3 = height
x4 = width / 2
y4 = height

rectangle_points = np.array([[(x1, y1), (x2, y2),
                              (x3, y3), (x4, y4)]], dtype=np.int32)
mask = cv2.fillPoly(mask, rectangle_points, 255)
masked_image = cv2.bitwise_and(im, mask)

cv2.imshow('originalna slika', im)
cv2.imshow('maska', mask)
cv2.imshow('maskirana slika', masked_image)
cv2.waitKey()
cv2.destroyAllWindows()
```



Maskiranje slike

Primjer - Maskiranje slike - ellipse

```
import cv2
import numpy as np

HEIGHT = 500
WIDTH = 700

mask = np.zeros((HEIGHT, WIDTH))

cv2.ellipse(img=mask,
            center=(WIDTH // 2, HEIGHT // 2),
            axes=(WIDTH // 2, HEIGHT // 2),
            angle=0,
            startAngle=180,
            endAngle=360,
            color=255,
            thickness=2)  # ako postavimo vrijednost na -1, elipsa bude popunjena

cv2.imshow("Mask", mask)
cv2.waitKey()
cv2.destroyAllWindows()
```



Video petlja

Video petlja

```python
import cv2

#capture = cv2.VideoCapture(0) -> video s web kamere
capture = cv2.VideoCapture('autocesta_video.mp4')

if not capture.isOpened():
    print('Unable to open video')
    exit(1)

while True:
    ret, frame = capture.read()

    if frame is None:
        break

    cv2.imshow('Frame', frame)

    if cv2.waitKey(100) == ord('q'):
        break


capture.release()
cv2.destroyAllWindows()
```

Detekcija linija voznih traka u videu

Detekcija linija voznih traka u videu

Algoritam:

1. Učitaj video
2. Za svaki video frame:
   a. Obradi sliku (median blur, pretvaranje u grayscale, Canny)
   b. Primijeni masku na područje interesa
   c. Houghovom transformacijom detektiraj linije
   d. Izračunaj nagibe detektiranih linija, te podijeli linije na lijeve i desne
   e. Izračunaj prosječne parametre lijevih i desnih linija
   f. Nacrtaj lijevu i desnu liniju na temelju izračunatih prosječnih vrijednosti

Detekcija linija voznih traka u videu

Rezultat:

Pitanja ?

Slijedi rješavanje zadataka